

5. Stacks and Queues

5.1. Basic Stack Operations

5.2. Basic Queue Operations

5.3. Implementation of Stacks and queues

The Stack

Stack

- **Linked list and arrays** allow one to insert and delete elements at any place in the list
 - At the beginning
 - At the end or
 - In the middle
- There are frequent situations where one wants to **restrict** insertions and deletions so they can take place only at the **beginning or the end of the list, not in the middle**
- Two of the data structures that are useful in such situations are
 - **stacks and**
 - **queues**

Stack

- A stack is a **linear data structure** (a list of elements) that has restricted data access
- Items (or elements) may be **added** (or **inserted**) and removed only **at one end**
- Is an ordered collection of entries that can only be accessed at one end
- Is a data structure of ordered entries such that entries can only be inserted and removed at one end (called **TOP**)
- When we say a stack is ordered we mean that there is one that can be accessed first (the one on the top), that can be accessed second (just below the top), the third one and so forth

Stack

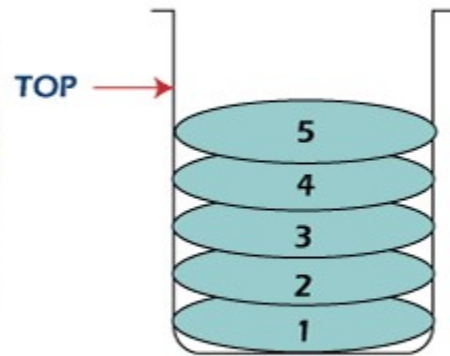
- It is a data structure that has access to its data only **at the end** (LL) or TOP (array)
- It operates on **Last-In First-Out (LIFO)** basis
 - The last item to be added to a stack is the first item to be removed
 - This means, elements are removed from the stack in the reverse order of that in which they were inserted into the stack

Stack

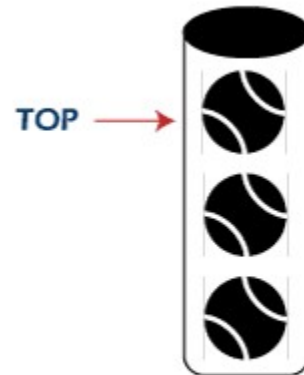
- Every day examples of such structures
 - Stack of dishes
 - Stack of pennies
 - Stack of folded towels
 - Stack of books
- An item may be added or removed from the top of any of the above stacks



Stack of Coins



Stack of Plates



Can of Tennis Balls



Stack of Books

Stack

- Other names used for stacks
 - Piles
 - Push down lists
- A stack has many applications in computer science
 - To be addressed later

Stack

- A stack has two basic operations, push and pop
- **push (or put)**
 - Is a term used for inserting or adding data (or element) at the top of the stack or into the stack
 - Could be written as Is a function to add an entry at the top of the stack
 - Needs at least one parameter

- **pop (get)**
 - Is a term used to delete or remove an element or data from the top of the stack
 - Could be written as a function to remove the top entry
 - No need to define the element to be removed in case of **pop** because there is only one element to be removed, the element at the top
 - No need to use parameter for **pop** operation
 - By default **pop** operation takes (or removes) the top element
 - Is the synonym for delete when it comes to stack

Stack

- There are no functions that allow a program to access entries other than the top entry
- To access any entry other than the top one, the program must remove entries one at a time from the top until the desired entry is reached
- **Include examples of stacks**
 - Include example 1, 2, and 3

- Stacks can be implemented both as
 - An array (contiguous list) and
 - Linked lists
- There are a set of operations that will work with either type of implementation (push, pop, peek, isEmpty, isFull, sizeOfStack)

Array representation of Stacks

- The array acts as a stack
- Let the array be called **STACK**
- Let **MAXSTK**
 - be the maximum number of elements that can be held by the stack
 - Capacity of the stack
 - Array implementation requires this constant because it uses a fixed size array to hold the items of the stack
 - The linked list implementation will use dynamic memory instead and such a constant is not required

Array representation of Stacks

- Suppose the stack has the following data structure (or definition)

```
int STACK[MAXSTK];
```

- We should have a an integer variable (or pointer variable) that points to the top element in the stack
- Call this variable **TOP**
- This implies TOP will contain the **index of the top element of the stack**

Array representation of Stacks

- At the beginning the **TOP variable (or pointer) must be initialized with -1**
- That is, we have no element in the stack at the beginning

int TOP = -1;

- TOP also tells us the total number of elements in the STACK
- How do you calculate the total number of elements in the stack using TOP?
 - Total number of elements in the stack = $\text{TOP} + 1$;
 - That is, $\text{TOP} + 1$ is the total number of elements in the stack

- **Stack overflow**
 - This is the condition resulting from trying to push (or add) an item to a full stack
 - To avoid stack overflow, compare the capacity constant MAXSTK to the stack's current size obtained with the size function

Stack Errors

- **Stack Underflow**

- The condition resulting from trying to POP (remove) an item from an empty stack
- If a program attempts to POP an item of empty stack, then it is called stack underflow
- In order to avoid a stack underflow, it is possible to write a function that tests whether the stack is empty or not

To push or store an element into the stack

- do the following
 - Check if there is enough space
 - In order to store data
 - TOP must be $\leq \text{MAXSTK} - 2$ or
 - TOP must be $< \text{MAXSTK} - 1$
 - because to store on the top, the top must be empty
 - If **yes** increment TOP and then store the element in `STACK[TOP]`
 - (incrementing must be done before storing/pushing the element)
 - If **no**, No space in the stack and send an overflow message (stack overflow)
- **We use the operation push**

push Operation

- Is a function to push or store an element into the stack
- There is nothing (i.e., data) to be returned by this function because it does only push element
- Thus, it must be of type void

push Operation

- Algorithm
 - Step 1 - increment the stack TOP by 1
 - i. Check whether stack is always less than upper limit of the stack
 - ii. If it is less than the upper limit go to step 2 else report “Stack Overflow”
 - Step 2 - put the new element at the position pointed by TOP

push Operation

`/* This procedure pushes an ITEM on to a stack*/`

`void push(STACK, TOP, MAXSTK, ITEM)`

`{`

`// Is stack already filled?`

`If(TOP==MAXSTK-1)`

`cout<<"\nOVERFLOW" //print overflow and return`

`else`

`{`

`TOP=TOP + 1; // TOP is set to increase by 1. The value of TOP, in case of
 push, is changed before the insertion in push`

`STACK[TOP]=ITEM               //insert ITEM in a new TOP position`

`}`

`return;`

`}`

- Remark
 - Frequently, TOP and MAXSTK are global variables and hence procedure push can be called using only **push(STACK, ITEM)**
 - If the array STACK is also declared globally, the procedure push can be called using only **push(ITEM)**

- C++ implementation
 - Include F
 - See stack2.cpp
 - See stackz.cpp

- do the following
 - Check if there is data in the stack (how?)
 - Check if $TOP \geq 0$
 - **If yes**
 - There is at least one element
 - Copy/remove the element in $STACK[TOP]$
 - Then decrement the value of the TOP
 - **If NO**
 - No element in the stack
 - Send stack underflow message
- We use the operation `pop()`

pop Operation

- The function **pop** has to return a data, top element, because the element may be used somewhere else
- The data type of the **pop** function is the same as the type of the STACK itself, in this case the type of the array

- Algorithm for pop() function
 - Step 1
 - if the stack is empty
 - give the alert “Stack Underflow!” and **quit**
 - else
 - go to step 2
 - Step 2
 - i. Hold the value for the element pointed by the TOP
 - ii. Put a NULL value instead
 - iii. Decrement the TOP by 1

/* This procedure deletes the top element of the stack and assigns it to the variable ITEM*/

DataType pop(STACK, TOP, ITEM)

```
{
    // Stack has item to be removed?
    If(TOP==-1)
        cout<<"\nUNDERFLOW" ; //print UNDERFLOW and return (TOP);
    else
    {
        ITEM=STACK[TOP];           // Assign (set) top element to ITEM
        STACK[TOP]=NULL;
        TOP=TOP - 1;//TOP is decreased by 1. The value of TOP, in case //of
        pop, is changed after the deletion in pop
    }
    return (ITEM);
}
```

pop Operation

- Remark
 - Frequently, TOP and MAXSTK are global variables and hence procedure pop can be called using only pop(**STACK, ITEM**)
 - If the array STACK is also declared globally, procedure pop can be called using only **pop(ITEM)** or simple **pop()**

pop Operation

- C++ implementation
 - Include G
 - Include stack2.cpp (second version)

isEmpty() & isFULL()

- We could have other operations, such as an explicit boolean function
 - isEmpty()- that tests for empty stack
 - isFull()- that tests for a full stacktogether with the pop and push functions
- **Include definitions of isEmpty() and isFull()**
 - **Include D**

peek()

- Sometimes we need information about the last element without removing the element
- For this we use the function **peek()** or also called **topData()**
- Include the definition for **topData()**
 - **Include E**

sizeofStack()

- Function to know the current size of the stack

//Returns the number of elements in the stack

```
int sizeofStack()
```

```
{
```

```
    /*post condition: return value is total number of items  
in the stack */
```

```
    return (TOP + 1);
```

```
}
```

Linked List Implementation of Stack

- A linked list is a natural way to implement a stack as a dynamic structure whose size can grow and shrink during execution, without a predefined limit that is determined at compilation
- The items in the stack are stored in a linked list, with the **TOP of the stack stored at the head node**, down to the bottom of the stack at the final node
- In case of array implementation, we have a variable TOP to indicate the top element

Linked List Implementation of Stack

- In this case, we need two pointers, one pointing to the beginning node (because we need to have a head pointer) of the list and the other the top of the list or node (because operation is performed at the top)
- At the beginning, these two pointers must be initialized to NULL

```
node * headPtr=NULL;  
node *topPtr=NULL;
```

Defining the Structure

```
struct number {  
    int num;  
    number *next;  
}  
number * bottomPtr =NULL;  
number * topPtr=NULL;
```

To push an element

- Check if there is a free space
 - How is this done?
 - We need to have a pointer **newNumPtr** that points to the newly created node
 - `number * newNumPtr=new number;`
 - This is used to reserve new space for our data.
 - How do we know that this space is reserved?
 - **Is newNumPtr != NULL?**
 - If != NULL, it means we have memory, memory is allocated and this memory is pointed by newNumPtr and its address is stored in newNumPtr
 - If the value stored at newNumPtr is NULL we have no space

- If `newNumPtr != NULL` is true, then
 - Store data in the newly created node (How?)
 - Create a link between the last node and the newly created node.
 - That is, the next pointer of the last node in the list should point to the newly created node
 - Make the top pointer to point to the last node
- If `newNumPtr != NULL` is false
 - Print “Memory overflow “

Linked list implementation of stacks: push operation

- Algorithm
 - Step 1- if the stack is empty, go to step 2 or else go to step 3
 - Step 2- create the new element and make bottomPtr and topPtr point to the new element and quit
 - Step 3- create the new element and make the last (top most) element of the stack to point to it
 - Step 4- make the new element your top most element by making the topPtr points to it

Linked list implementation of stacks: push operation

- Include the C++ code
 - Include H
 - See stack5.cpp

Linked list implementation of stacks: push operation

- **Remarks**

- The push operation is similar to the insertion operation in a dynamic singly linked list
- The only difference here is that you can add the new element only at the end of the list, which means additions can happen only from the TOP
- Since dynamic list is used for the stack, the stack is also dynamic, means it has no prior upper limit
- So there is no need to check for the overflow condition at all!

To pop an element

- Check if there is data in the stack
- How do you check whether there is an element in the list or stack?
 - Is bottomPtr != NULL; (ask this question)

To pop an element

- **If bottomPtr != NULL is true**
 - bottomPtr is different from NULL
 - That is, There is data in the list (at least one)
- **If bottomPtr != NULL is false**
 - bottomPtr=NULL
 - Stack is empty
 - There is no element in the stack to pop
 - Print “Stack Underflow”
- **Note: 1. topPtr always points to the last node**
 - 2. We need a pointer prevNumPtr to point to the previous node of the last node

number * prevNumPtr;

Linked list implementation of stacks: pop operation

- ### Algorithm

1. If the stack is empty then give an alert message “stack Underflow” and quite; or else proceed
2. If there is only one element left go to step 3 or else step 4
3. Free that element and make bottomPtr and topPtr to point to NULL and quit
4. Make “targetPtr” point to just one element before top;; free the top most element ; make targetPtr to point to the top most element

Linked list implementation of stacks: pop operation

- Include the C++ code
 - Include l
 - See stack5.cpp

Linked list implementation of stacks: pop operation

- The pop operation is similar to the deletion operation in any linked list but you can only delete from the end of the list and only one at time
- Here, we need a list pointer, targetPtr, which will be pointing to the last but one element in the list (stack)
- Every time we pop, the TOP most element will be deleted and targetPtr will be made as the top most element
- Supposing you have only one element left in the stack, then we won't make use of targetPtr. Rather we will take help of bottomPtr

Stack Applications

- Surprisingly, stacks have many application
- Most compilers use stacks to analyze the syntax of a program
- Stacks are used
 - To keep track of local variables when a program is run
 - To search a maze or family tree or other types of branching trees

Applications of Stacks

Infix to Postfix (RPN) Conversion

Types of Expression

- **Notations for Arithmetic Expression**
- There are three notations to represent an arithmetic expression:
 - **Infix Notation**
 - **Prefix Notation**
 - **Postfix Notation**
- The normal (or human) way of expressing mathematical expressions is called **infix** form, e.g. $4+5*5$.

Prefix Notation(Polish notation)

- The prefix notation places the operator before the operands. This notation was introduced by the Polish mathematician and hence often referred to as polish notation.

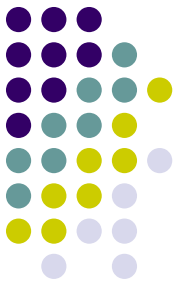
Example : Infix : $4+5*5$

- **the prefix form: + * 4 5 5**
- When the operators come after their operands, it is called **postfix form**

Postfix Notation(Reverse Polish notation)

- The postfix notation places the operator after the operands.
- This notation is just the reverse of Polish notation and also known as **Reverse Polish notation**.

e.g. 4 5 5 * +



Transforming Infix to Postfix

- Goal is show you how to evaluate infix algebraic expression, postfix expressions are easier to evaluate.
- So first look at how to represent an infix expression by using postfix notation.

Infix

Postfix

$a+b$

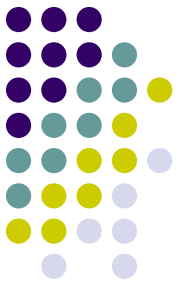
$ab+$

$(a+b)*c$

$ab+c*$

$a + b*c$

$abc*+$



Pencil and paper scheme

- We write the infix expression with a fully parenthesized infix expression.
- For example: $(a+b)*c$ as $((a+b)*c)$. Each operator is associated with a pair of parentheses.
- Now we move each operator to the right, so it appears immediately before its associated close parenthesis:
- $((ab+)c^*)$
- Finally, we remove the parentheses to obtain the postfix expression. $ab+c^*$

infix Notation Prefix Notation

$$A * B$$

* A B

$$(A+B)/C$$

$\div + ABC$

$$(A * B) + (D - C)$$
$$+^*AB - DC$$

Postfix Notation

AB*

 $AB+C/$

AB*DC-+

Exercise

a + b * c

$$a * b / (c - d)$$
$$a / b + (c - d)$$
$$a / b + c - d$$

Answers

abc^*+

$ab^*cd- /$

$ab/cd-+$

$ab/c+d-$

Infix to Postfix (RPN) Conversion

- Scan the infix expression from left to right.
- Encounter an **operand**, place it at the end of new expression.
- Encounter an **operator**, save the operator until the right time to pop up
 - Depends on the relative precedence of the next operator.
 - **Higher precedence operator can be pushed**
 - **Same precedence, operator gets popped.**
- Reach the end of input expression, we pop each operator from the stack and append it to the end of the output

Transforming Infix to Postfix

Next Character	Postfix	Operator Stack (bottom to top)
<i>a</i>	<i>a</i>	
<i>+</i>	<i>a</i>	<i>+</i>
<i>b</i>	<i>a b</i>	<i>+</i>
<i>*</i>	<i>a b</i>	<i>+</i> <i>*</i>
<i>c</i>	<i>a b c</i>	<i>+</i> <i>*</i>
	<i>a b c *</i>	<i>+</i>
	<i>a b c * +</i>	

Converting the infix expression

a + b * c to postfix form

Converting infix expression to postfix form:

$$a - b + c$$

Next Character	Postfix	Operator Stack (bottom to top)
<i>a</i>	<i>a</i>	
<i>−</i>	<i>a</i>	<i>−</i>
<i>b</i>	<i>a b</i>	<i>−</i>
<i>+</i>	<i>a b −</i>	
	<i>a b −</i>	<i>+</i>
<i>c</i>	<i>a b − c</i>	<i>+</i>
	<i>a b − c +</i>	

Infix to Postfix (RPN) Conversion

Algorithm

initialize stack and postfix output to empty;

while(not end of infix expression) {

 get next infix **item**

if(item is operand) {

append item to postfix output

}

else if(item == '(') {

push item onto stack

}

else if(item == ')') {

pop stack to x

while(x != '(') {

 app.x to postfix output &

 pop stack to x

}

```
else {
```

```
    while(precedence(stack top) >= precedence(item)){
```

```
        pop stack to x ;
```

```
        app.x to postfix output
```

```
    }
```

```
    push item onto stack
```

```
}
```

```
}
```

```
while(stack not empty){
```

```
    pop stack to x ;
```

```
    append x to postfix output
```

```
}
```

Operator Precedence (for this algorithm):

4 : '(' - only popped if a matching ')' is found

3 : All unary operators

2 : / *

1 : + -

Converting infix expression to postfix form:

$a \wedge b \wedge c$

An exception: if an operator is $\wedge$, push it onto the stack regardless of what is at the top of the stack

Next Character	Postfix	Operator Stack (bottom to top)
a	a	
$\wedge$	a	$\wedge$
b	$a b$	$\wedge$
$\wedge$	$a b$	$\wedge \wedge$
c	$a b c$	$\wedge \wedge$
	$a b c \wedge$	$\wedge$
	$a b c \wedge \wedge$	

- **Infix-to-Postfix Algorithm**

Symbol in Infix	Action
Operand	Append to end of output expression
Operator ^	Push ^ onto stack
Operator +, -, *, or /	Pop operators from stack, append to output expression until stack empty or top has lower precedence than new operator. Then push new operator onto stack
Open parenthesis	Push (onto stack, treat it as an operator with the lowest precedence
Close parenthesis	Pop operators from stack, append to output expression until we pop an open parenthesis. Discard both parentheses.

Steps to convert the infix expression

$a / b * (c + (d - e))$ to postfix form.

Next Character	Postfix	Operator Stack (bottom to top)
a	a	
$/$	a	$/$
b	$a b$	$/$
$*$	$a b /$	$*$
	$a b /$	$*$
$($	$a b /$	$* ($
c	$a b / c$	$* ($
$+$	$a b / c$	$* (+$
$($	$a b / c$	$* (+ ($
d	$a b / c d$	$* (+ ($
$-$	$a b / c d$	$* (+ (-$
e	$a b / c d e$	$* (+ (-$
$)$	$a b / c d e -$	$* (+ ($
	$a b / c d e -$	$* (+$
$)$	$a b / c d e - +$	$* ($
	$a b / c d e - +$	$*$
	$a b / c d e - + *$	

- $(a + b)/(c - d)$
- $a / (b - c) * d$
- $a - (b / (c - d) * e + f) ^ g$
- $(a - b * c) / (d * e ^ f * g + h)$

Answer

$ab+cd-/$

$abc-/d^*$

$abcd-/e^*f + g^{\wedge}-$

$abc^* -def^{\wedge}*g^*h+ /$

Example:

Convert to postfix expression:

a) $4 + 3 * (6 * 3 - 12)$

b) $A + B * C + D$

c) $(A + B) * (C + D)$

d) $A + (B * C + D) / E$

e) $10 + 3 * 5 / (16 - 4)$

Postfix Evaluation

Consider the postfix expression :

6 5 2 3 + 8 * + 3 + *

Evaluating Postfix Expression

- Save operands until we find the operators that apply to them.
- When see operator, is second operand is the most recently save value. The value saved before it – is the operator's first operand.
- Push the result into the stack. If we are at the end of expression, the value remains in the stack is the value of the expression.

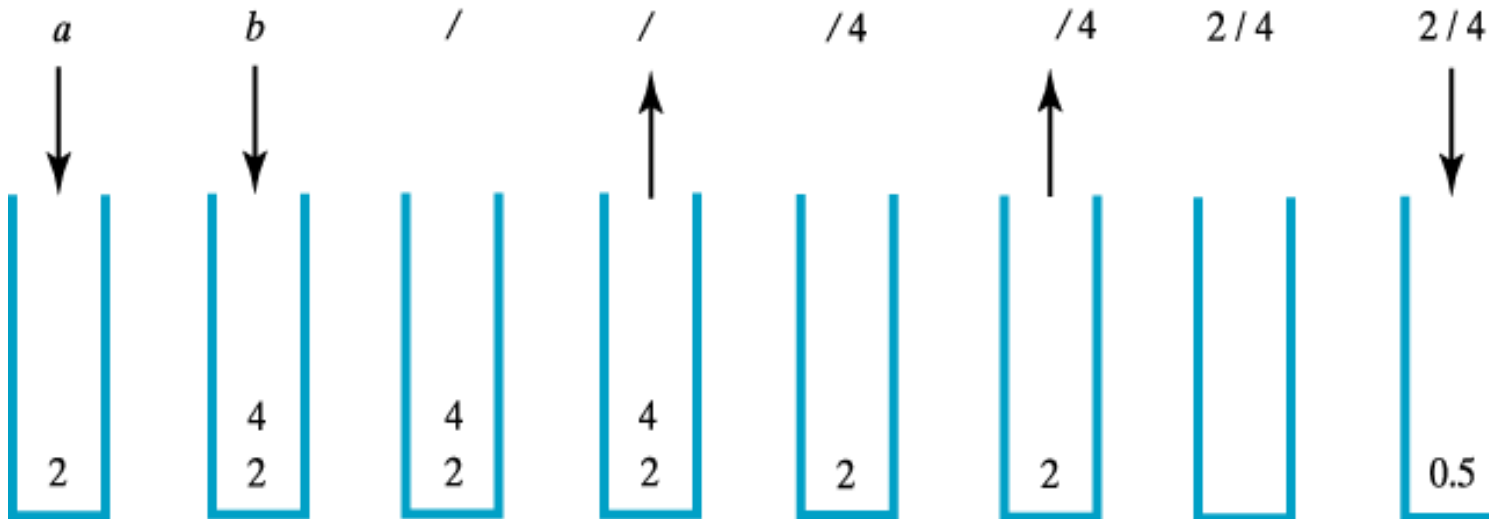
Algorithm

```
initialise stack to empty;  
while (not end of postfix expression) {  
    get next postfix item;  
    if(item is value)  
        push it onto the stack;  
    else if(item is binary operator) {  
        pop the stack to x;  
        pop the stack to y;  
        perform y operator x;  
        push the results onto the stack;  
    }  
    else if (item is unary operator) {  
        pop the stack to x;  
        perform operator(x);  
        push the results onto the stack  
    }  
}
```

The single value on the stack is the desired result.

- The stack during the evaluation of the postfix expression $a \ b \ /$ when a is 2 and b is 4

Evaluating Postfix Expression



Binary operators: +, -, *, /, etc.,

Unary operators: unary minus, square root, sin, cos, exp, etc.,

Example:

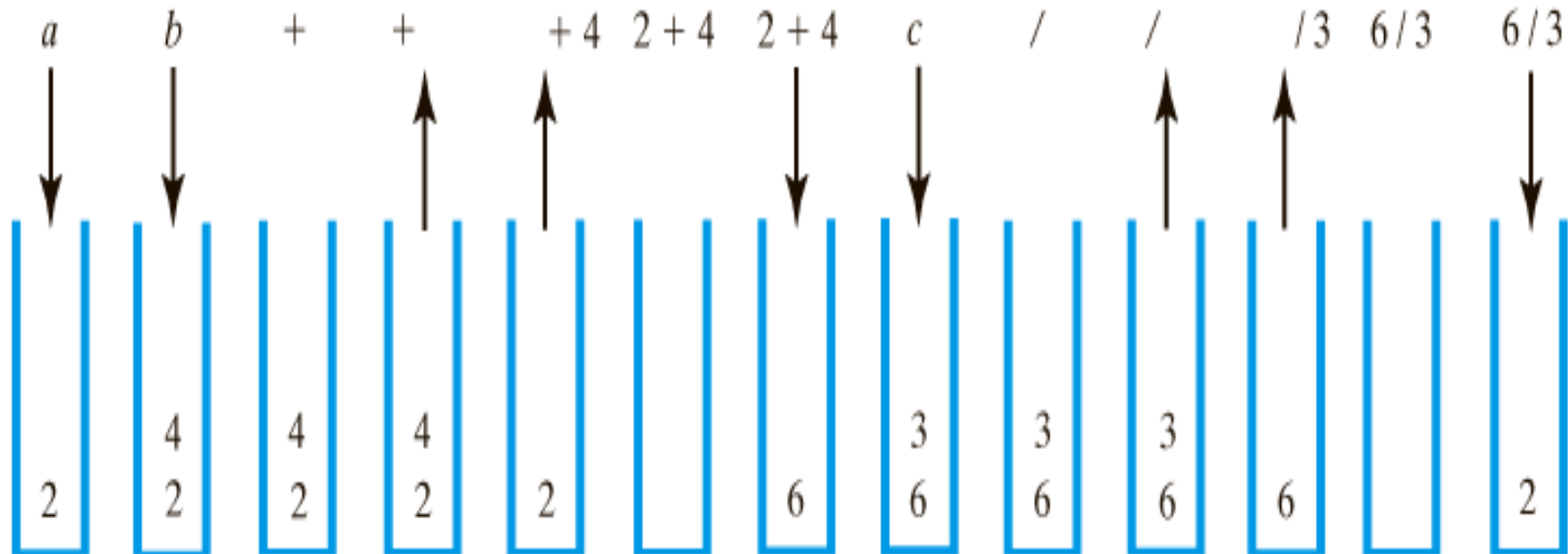
Evaluate:

a) 6 5 2 3 + 8 * + 3 + *

b) 4 5 + 7 2 - *

c) 4 2 3 5 1 - + * +

The stack during the evaluation of the postfix expression $a \ b \ + \ c \ /$ when a is 2, b is 4 and c is 3



Exercise:

- Evaluate the following postfix expression.
Assume that $a=2$, $b=3$, $c=4$, $d=5$, and $e=6$
- $ae+bd-/$
- abc^*d^*-
- $abc-/d^*$
- $ebca^{*+}d-$

Application of stack:

Balanced parentheses using Stack

Here are some of the balanced and unbalanced expressions:

BALANCED EXPRESSION	UNBALANCED EXPRESSION
$(a + b)$	$(a + b$
$[(c - d) * e]$	$[(c - d * e]$
$\{()\}[]$	$\{[(())]\}$

Steps to find whether a given expression is balanced or unbalanced

1. Input the expression and put it in a character stack.
2. Scan the characters from the expression one by one.
3. If the scanned character is a starting bracket (' (' or ' { ' or ' ['), then push it to the stack.
4. If the scanned character is a closing bracket (') ' or ' } ' or '] '), then pop from the stack and if the popped character is the equivalent starting bracket, then proceed. Else, the expression is unbalanced.
5. After scanning all the characters from the expression, if there is any parenthesis found in the stack or if the stack is not empty, then the expression is unbalanced.

Step 1:

Stack []

str [{ () }]

Opening bracket. Push into stack



Step 2:

Stack [{]

str [{ () }]

Opening bracket. Push into stack



Step 3:

Stack [{ (]

str [{ () }]

Closing bracket. Check top of the stack if same kind or not



Step 4:

Stack [{]

str [{ () }]

Closing bracket. Check top of stack if same kind or not



Stack in function calls

- **Program Stack:** Program Stack is the stack which holds all the function calls, with bottom elements as the main function.
- **Stack Frame:** Stack Frame is actually a buffer memory that is an element of program stack and has data of the called function ie
 - Return Address
 - Input Parameter
 - Local Variables
- whenever a function is called a **new stack frame** is created with all the function's data and this **stack frame is pushed in the program stack**

Series of operations when we call a function:

1. Stack Frame is pushed into stack.
2. Sub-routine instructions are executed.
3. Stack Frame is popped from the stack.
4. Now Program Counter is holding the return address.

Queue

Queue

- Is an ordered collection of items from which items may be **deleted at one end** (called **front** of the queue) and into which items may be **inserted at the other end** (called the **rear** of the queue)
- That is, it is a data structure that has access to its data at the front and rear

Queue

- It is a First-In-First-Out (FIFO) data structure
 - That is, the first element inserted into a queue is the first element to be removed
 - The order in which elements enter a queue is the order in which they leave
 - This is in contrast with the queue

Queue

- Examples of a queue in the real world
 - people waiting in line at the bank form a queue where the first person in line is the first person to be waited on
 - A line at the bus stop
 - Automobiles waiting to pass through an intersection (form a queue) in which the first car in line is the first car through
 - A group of cars waiting at a toll booth

Queue

- Has three primitive operations applied to it
 - **enqueue(q,x)** also called insert(q,x)
 - Inserts item (data) x at the rear of the queue q
 - **X=dequeue(q)** also called remove(q)
 - Deletes the front element (data) from the queue q and sets X to its content
 - **empty(q)**
 - Returns false or true depending on whether or not the queue contains any elements

Queue

- **Example**

Operation	Content of queue
Enqueue(q, B)	
Enqueue(q, C)	
Enqueue(q)	
Enqueue(q, G)	
Enqueue(q, F)	
Enqueue(q)	
Enqueue(q, A)	
Enqueue(q)	

Queue

- Two ways to represent (or implement) a queue
 - **As an array**
 - Simple array representation of queue
 - Circular array representation of queue
 - **As a linked list**

Simple array representation of queue

- Array is used to hold the elements
- We need two integer variables, **FRONT** and **REAR**, to hold the **positions** with in the array of the **first** and last **elements** of the queue
 - **FRONT** tells (or holds) the index of the front element
 - **REAR** tells (or holds) the index of the rear element
- We also need the following integer variables
 - **QUEUE_SIZE**
 - tells (or holds) the total number of data in the queue
 - **MAX_SIZE**
 - tells (or stores) the capacity of the array (i.e. queue)

Simple array representation of queue

- Initially FRONT and REAR must be initialized to -1

int FRONT = -1, REAR = -1;

- Initially **QUEUESIZE** must be initialized to 0

int QUEUESIZE = 0

- Remark
 - Using an array to hold a queue introduces the possibility of overflow if the queue should grow larger than the size of the array

Simple array representation of queue

- Declaring a queue q of integers

```
#define MAXQUEUE 100
Struct queue{
    int items[MAXQUEUE];
    int front, rear;
}q;
```

- **Algorithm to enqueue data to the queue**
 - Check if there is space in the queue
 - To do this ask this question
 - **Is $REAR < MAX_SIZE - 1$?**
 - If $(REAR < MAX_SIZE - 1)$ is **true**
 - Increment $REAR$ by 1
 - Store the data in $queue[REAR]$
 - **Increment $QUEUESIZE$ by 1**
 - If $(FRONT == -1)$
Increment $FRONT$ by 1
 - If $(REAR < MAX_SIZE - 1)$ is **false**
 - Display the message **“Queue Overflow”**

Simple array representation of queue

- **Algorithm to dequeue data from the queue**
 - Check if there is data in the queue
 - To do this ask this question
 - Is $QUEUESIZE > 0$?
 - If $(QUEUESIZE > 0)$ is **true**
 - Copy the data in `queue[FRONT]`
 - Increment `FRONT` by 1
 - Decrement `QUEUESIZE` by 1
 - If $(QUEUESIZE > 0)$ is **false**
 - Display the message “Queue Underflow”

C++ implementation of enqueue and dequeue

```
const int MAX_SIZE=100;
```

```
int FRONT =-1, REAR =-1;
```

```
int QUEUE_SIZE = 0;
```

C++ implementation of enqueue and equeue

```
void enqueue(char queue[], char x)
{
    if(REAR < MAX_SIZE - 1)
    {
        REAR++;
        queue[REAR] = x;
        QUEUESIZE++;
        if(FRONT == -1)
            FRONT++;
    }
    else
        cout<<"Queue Overflow";
}
```

C++ implementation of enqueue and equeue

```
int dequeue(char queue[])
{
    char x;
    if(QUEUE_SIZE>0)
    {
        x=queue[FRONT];
        FRONT++;
        QUEUE_SIZE--;
    }
    else
        cout<<"Queue Underflow";

    return  (x);
}
```

Array representation of queue

- Example 1
 - Consider a queue with `QUEUE_SIZE = 4` and write a C++ program that performs the operations shown in the table given in the next page
 - Answer
 - See `queue3.cpp`

Simple array representation of queue

Operation	Simple array			
	Content of the array	Content of the Queue	QUEUE SIZE	Message
Enqueue(B)	B	B	1	
Enqueue(C)	B C	BC	2	
Dequeue()	C	C	1	
Enqueue(G)	C G	CG	2	
Enqueue (F)	C G F	CGF	3	
Dequeue()	G F	GF	2	
Enqueue(A)	G F	GF	2	Overflow
Enqueue(D)	G F	GF	2	Overflow
Enqueue(C)	G F	GF	2	Overflow
Dequeue()	F	F	1	
Enqueue(H)	F	F	1	Overflow
Dequeue ()		Empty	0	
Dequeue()		Empty	0	Underflow
Dequeue()		Empty	0	Underflow
Dequeue()		Empty	0	Underflow
Dequeue()		Empty	0	Underflow

Limitation of simple array representation of queue

- The array representation outlined earlier has a limitation
 - The variable **rear** is incremented but never decremented
 - Hence, it will quickly reach the end of the array
 - At that point, we will not be able to add any more elements to the queue, yet there is likely to be room in the array
 - In fact, it is possible to reach the absurd situation where the queue is empty, yet no new element can be inserted
 - In normal application, the variable front would also be incremented from time to time (when entries are removed from the queue)
 - This will free up the array application with index values less than **front**
- Different solutions to overcome this problem

Solution 1

- For using the freed array locations
 - Maintain all the queue entries so that front is always equal to 0 (the first index of the array)
 - When `queue[0]` is removed, we move all the entries in the array down one locations, so that the value of `queue[1]` is moved to `queue[0]`, and then all other entries are also moved
- How is the above method implemented (see next slide)

Solution 1

- Modify the dequeue operation so that when an item is deleted, the entire queue is shifted to the beginning of the array
- Ignoring the possibility of overflow, the operation `x=dequeue(queue)` would be modified as follows

```
x=queue(FRONT);  
for(int i = 0; i < REAR; i++)  
{  
    queue[i] = queue[i+1];  
}  
REAR--;
```

Solution 1 ...

- **The queue need no longer contain a front field**
- **This is because**, the element at position 0 of the array is always at the front of the queue
- **The empty queue is represented by the queue in which rear is -1**

Solution 1 ...

- **Is this method efficient?**
 - The method will work
 - **But, it is not efficient, in fact too inefficient**
 - Each deletion involves moving every remaining element of the queue
 - **Every time we remove an entry from the queue, we must move every entry in the queue**
 - If a queue contains 500 or 100 elements, this is high a price to pay
 - Further, the operation of removing an element from a queue logically involves manipulation of only one element
 - The implementation of that operation should reflect this and should not involve a host of extraneous operations

Solution 1 ...

- Exercise
 - Include exercise 4.1.3

Solution 2

- Is a better approach
- We do not need to move all the array elements
- When the rear index reaches the end of the array, we can simply start using the variable locations at the front
- One way to think of this arrangement is to think of the array as being bent into a circle so that the first component of the array is immediately after the last component of the array
- That is, the successor of the last array index is the first array index

Solution 2...

- In this arrangement (circular queue), the free index positions are always to the “right after” `queue[REAR]`
- Example
 - Consider a queue of characters with a capacity of 4
 - Perform the operations given in the table in the next slide considering the queue is represented as circular array rather than simple array
 - See also D

Solution 2 ...

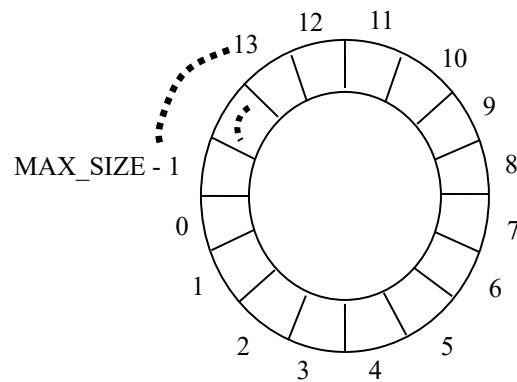
Operation	Simple array				Circular array			
	Content of the array	Content of the Queue	QUEUE SIZE	Message	Content of the array	Content of the queue	QUEUE SIZE	Message
Enqueue(B)	B	B	1		B	B	1	
Enqueue(C)	B C	BC	2		B C	BC	2	
Dequeue()	C	C	1		C	C	1	
Enqueue(G)	C G	CG	2		C G	CG	2	
Enqueue (F)	C G F	CGF	3		C G F	CGF	3	
Dequeue()	G F	GF	2		G F	GF	2	
Enqueue(A)	G F	GF	2	Overflow	A G F	GFA	3	
Enqueue(D)	G F	GF	2	Overflow	A D G F	GFAD	4	
Enqueue(C)	G F	GF	2	Overflow	A D G F	GFAD	4	Overflow
Dequeue()	F	F	1		A D F	FAD	3	
Enqueue(H)	F	F	1	Overflow	A D H F	FADH	4	
Dequeue ()		Empty	0		A D H	ADH	3	
Dequeue()		Empty	0	Underflow	D H	DH	2	
Dequeue()		Empty	0	Underflow	H	H	1	
Dequeue()		Empty	0	Underflow		Empty	0	
Dequeue()		Empty	0	Underflow		Empty	0	Underflow

Solution 2 ...

- With this view, the queue's entries start at `queue[FRONT]` and continue forward
 - If you reach the end of the array, then come back at `queue[0]`, and keep going until you find the REAR
 - It may help to actually view the array as bent circle, with the final array element attached back to the front as shown below
-
- 104
 - Include diagram and the statement just above it on kinfe
 - See also mu not
-
- An array used in this way is called a circular array (if we have free slot we can use it)

Solution 2 ...

- The circular array implementation of a queue with capacity `QUEUESIZ` can be simulated as follows:



- An array used in this way is called a circular array (if we have free slot we can use it)
- In any case, `queue[rear]` is the last entry in the queue

Circular array implementation of queue

- Consider the structure `queue[QUEUESIZE]`
- We need to have three integer variables, **FRONT**, **REAR** and **QUEUESIZE** and **MAX_SIZE**
 - REAR tells (stores) the index of the front element
 - FRONT tells (stores) the index of the rear element
 - QUEUESIZE tells (stores) the total number of data in the queue
 - MAX_SIZE tells (stores) the capacity of the queue
- Initially FRONT and REAR must be initialized to -1
`int FRONT = -1, REAR = -1;`
- Initially QUEUESIZE must be initialized to 0
`int QUEUESIZE = 0;`

Circular array implementation of queue

- **Algorithm to enqueue data to the queue**
 - Check if there is space in the queue
 - To do this ask this question
Is $QUEUE_SIZE < MAX_SIZE$?
 - If($QUEUE_SIZE < MAX_SIZE$) is **true**
 - Increment REAR by 1
 - If($REAR == MAX_SIZE$)
 $REAR = 0$
 - Store the data in $queue[REAR]$
 - Increment $QUEUE_SIZE$ by 1
 - If($FRONT == -1$)
Increment $FRONT$ by 1
 - If($QUEUE_SIZE < MAX_SIZE$) is false
display the message “Queue Overflow”

Circular array implementation of queue

- **Algorithm to dequeue data from the queue**
 - Check if there is data in the queue
 - To do this ask this question
Is $QUEUESIZE > 0$?
 - If($QUEUESIZE > 0$) is **true**
 - Copy the data in `queue[FRONT]`
 - Increment `FRONT` by 1
 - If($FRONT == QUEUESIZE$)
Set $FRONT = 0$
 - Decrement $QUEUESIZE$ by 1
 - If($QUEUESIZE > 0$) is **false**
Display the message “Queue Overflow”

C++ implementation of enqueue and dequeue (circular array of queue)

```
const int MAX_SIZE=100;  
int FRONT =-1, REAR =-1;  
int QUEUE_SIZE = 0;
```

C++ implementation of enqueue and dequeue (Circular array of queue)

```
void enqueue(int x)
{
    if(QUEUESIZE<MAX_SIZE)
    {
        REAR++;
        if(REAR == MAX_SIZE)
            REAR=0;
        Num[REAR]=x;
        QUEUESIZE++;
        if(FRONT == -1)
            FRONT++;
    }
    else
        cout<<"Queue Overflow";
}
```

C++ implementation of enqueue and equeue (Circular array of queue)

```
int dequeue()
{
    int x;
    if(QUEUESIZE>0)
    {
        x=Num[FRONT];
        FRONT++;
        if(FRONT == MAX_SIZE)
            FRONT = 0;
        QUEUESIZE--;
    }
    else
        cout<<"Queue Underflow";
    return(x);
}
```

C++ implementation of enqueue and dequeue (Circular array of queue)

- **Example 3**
 - Assume a queue is implemented as circular array
MAX_SIZE = 4 and write a C++ program that performs the operations shown in the table given in the next page
 - Answer
 - See queue5.cpp

C++ implementation of enqueue and equeue (Circular array of queue)

Operation	Simple array				Circular array			
	Content of the array	Content of the Queue	QUEUE SIZE	Message	Content of the array	Content of the queue	QUEUE SIZE	Message
Enqueue(B)	B	B	1		B	B	1	
Enqueue(C)	B C	BC	2		B C	BC	2	
Dequeue()	C	C	1		C	C	1	
Enqueue(G)	C G	CG	2		C G	CG	2	
Enqueue (F)	C G F	CGF	3		C G F	CGF	3	
Dequeue()	G F	GF	2		G F	GF	2	
Enqueue(A)	G F	GF	2	Overflow	A G F	GFA	3	
Enqueue(D)	G F	GF	2	Overflow	A D G F	GFAD	4	
Enqueue(C)	G F	GF	2	Overflow	A D G F	GFAD	4	Overflow
Dequeue()	F	F	1		A D F	FAD	3	
Enqueue(H)	F	F	1	Overflow	A D H F	FADH	4	
Dequeue ()		Empty	0		A D H	ADH	3	
Dequeue()		Empty	0	Underflow	D H	DH	2	
Dequeue()		Empty	0	Underflow	H	H	1	
Dequeue()		Empty	0	Underflow		Empty	0	
Dequeue()		Empty	0	Underflow		Empty	0	Underflow

C++ implementation of enqueue and equeue (Circular array of queue)

- Example 4
 - This is another example on a circular array implementation queue
 - See queue6.cpp

Linked list implementation of enqueue and dequeue operations

- Enqueue- is inserting a node at the end of a linked list
- Dequeue- is deleting the first node in the list
- **Define the structure** to hold references to queue nodes for the linked queue implementation

```
struct queue    {  
                char item;  
                queue next  
};
```

```
queue *frontPtr=NULL; //references to the front of the queue  
queue *rearPtr=NULL; //references to the rear of the queue
```

Linked list implementation of enqueue operation

/* Adds item to the rear of this queue */

void enqueue(char x)

{

queue * temp = new queue;

if(temp == NULL)

cout<<"\nQueue is FULL";

temp->item = x;

temp -> next = NULL;

to be checked

if(rearPtr == NULL)

frontPtr=temp;

else

rearPtr -> next = temp;

rearPtr = temp;

}

Linked list implementation of dequeue operation

/* Removes front element form this queue and return it */

void dequeue()

{

 char data;

 data = frontPtr -> item;

 frontPtr = frontPtr -> next;

 if(frontPtr == NULL)

 rearPtr = NULL;

 return data;

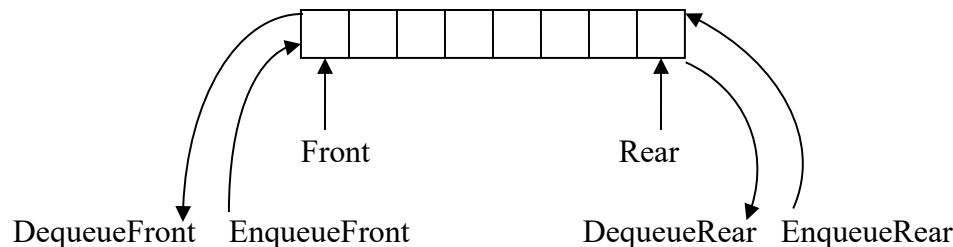
}

Linked list implementation of enqueue and dequeue operations

- Example
 - queue7.cpp
 - Queue8.cpp

Deque (pronounced as Deck)

- Is a **Double Ended Queue**
- A stack allows insertion and deletion of elements at only one end
- A queue allows insertion at one end and deletion at the other end
- A deque allows insertion and deletion at both ends
- insertion and deletion can occur at either end



Deque (pronounced as Deck)

- Has the following basic operations
 - EnqueueFront – inserts data at the front of the list
 - DequeueFront – deletes data at the front of the list
 - EnqueueRear – inserts data at the end of the list
 - DequeueRear – deletes data at the end of the list
- Implementation is similar to that of queue
- Is best implemented using doubly linked list

Deque (pronounced as Deck)

- Example
 - Queue9.cpp
- Exercise
 - How can a queue be represented as a C++ array?
 - Write four $O(1)$ -time procedures EnqueueFront, DequeueFront, EnqueueRear and DequeueRear to insert elements into and delete elements from both ends of a deque constructed from an array. Make sure that the functions work properly for the empty deque and that they detect overflow and underflow

Priority Queue

- Using a queue ensures that customers are served in the exact order in which they arrive
- However, we often want to assign priorities to customers and serve the higher priority customers before those of lower priority
- In fact, in many situations, simple queues are inadequate since first-in-first-out scheduling has to be overruled using some priority criteria
- Examples
 - A hospital emergency room will handle most severely injured patients first, even if they are not “first in line”
- In situations like this, a modified queue, or priority queue, is needed

Priority Queue

- Is a data structure that stores entries along with a priority for each entry
- Is a queue where each element has an associated key which indicates priority of the elements
- This key is provided at the time of insertion
- Entries are removed in order of priorities
- The highest priority entry is removed first
- If there are several entries with equal high priorities, then the one that was placed in the priority queue first is the one removed first

Priority Queue

- That is, in priority queues, elements are dequeued according to
 - Their priority and
 - Their current queue position
- Elements arrive randomly to the priority queue
- Thus, there is no guarantee that the front elements will be the most likely to be dequeued and the elements put at the end will be the last candidates for dequeuing

Priority Queue

- A wide spectrum of possible criteria can be used to prioritize elements in different cases
 - Frquency of use
 - Birth date
 - Salary
 - Postion
 - Status
 - gender

Priority queue enqueue and dequeue operations

- Dequeue operation deletes data having highest priority in the list
- One of the previously used dequeue or enqueue operations has to be modified
- **Example:**
 - Consider the following queue of persons where females have higher priority than males (gender is the key to give priority).

Abebe	Alemu	Aster	Belay	Kedir	Meron	Yonas
Male	Male	Female	Male	Male	Female	Male

Priority queue enqueue and dequeue operations

- **Dequeue() deletes Aster**

Abebe	Alemu	Belay	Kedir	Meron	Yonas
Male	Male	Male	Male	Female	Male

- **Dequeue() deletes Meron**

Abebe	Alemu	Belay	Kedir	Yonas
Male	Male	Male	Male	Male

- Now the queue has data having equal priority and dequeue operation deletes the front element like in the case of ordinary queues.

Priority queue enqueue and dequeue operations

- **Dequeue() deletes Abebe**

Alemu	Belay	Kedir	Yonas
Male	Male	Male	Male

- **Dequeue() deletes Alemu**

Belay	Kedir	Yonas
Male	Male	Male

- Thus, in the above example the implementation of the **dequeue operation need to be modified.**

Types of Priority queues

- There are two types of priority queues, **ascending priority queue** and **descending priority queue**

Ascending Priority Queue

- Is a collection of items into which items can be inserted arbitrarily and from which only the smallest item can be removed
 - The operation `enqueue(pq,x)` inserts element `x` into the `pq` and
 - The operation `dequeue(pq)` removes the minimum element from `pq` and returns its value the queue

Descending Priority Queue

- Is a collection of items into which items can be inserted arbitrarily and from which only the largest item can be removed
 - The operation `enqueue(pq,x)` inserts element `x` into the `pq` and is logically identical with the `enqueue(pq,x)` operation for an ascending priority queue
 - The operation `dequeue(pq)` removes the maximum element from `pq` and returns its value

Priority queue

- Examples
 - Queue10.cpp (array implementation)
 - Queue11.cpp (ascending priority)
 - Linked list implementation
 - Queue12.cpp (descending priority)
 - Linked list implementation
 - Queue13.cpp
 - Linked list implementation
 - Queue14.cpp
 - Linked list implementation
 - Queue15.cpp
 - Linked list implementation

Reading assignment

- Demerging queues
- Merging queues
- Application of queues

– Page 61

